Explainable Dual-Tier Guardrails for Robust Multilingual Prompt Injection Defense

Given Name Surname
Affiliation
City, Country
email@address.com

***Abstract*—As Large Language Models (LLMs) transition from research prototypes to production-grade agentic systems, the threat of prompt injection has emerged as a primary security bottleneck. Current defenses often prioritize detection accuracy over transparency and efficiency, resulting in "black-box" systems that are difficult to audit and prone to over-defense. This paper presents a robust, dual-tier guardrail framework designed for multilingual (English and Vietnamese) protection. Our framework combines a high-speed character-level baseline with a parameter-efficient transformer model fine-tuned via Low-Rank Adaptation (LoRA). We introduce an adversarial-robust normalization layer to mitigate character-level obfuscation and leverage Integrated Gradients for token-level risk attribution. Our evaluation on a diverse dataset of 33,346 samples demonstrates that the proposed system achieves state-of-the-art detection rates (99.8% F1 on Tier 1) while maintaining a low False Positive Rate (FPR), ensuring minimal impact on benign user interactions. We provide a rigorous analysis of the trade-offs between latency, accuracy, and interpretability, offering a blueprint for reproducible AI security in complex multilingual environments.**

***Index Terms*—Prompt Injection, LLM Security, XAI, LoRA, Multilingual NLP, Adversarial Robustness, AI Governance, RAG Security.**

I. Introduction

The rapid proliferation of Large Language Models (LLMs) has fundamentally transformed the landscape of human-computer interaction. From autonomous coding assistants to complex Retrieval-Augmented Generation (RAG) systems, LLMs are increasingly being integrated into critical business logic and decision-making processes. However, this deep integration has exposed a fundamental architectural flaw: the collapse of the instruction and data channels. In traditional computing, code and data are typically handled by separate execution and storage layers. In LLMs, both instructions (from developers) and data (from untrusted users or external sources) are processed by the same neural mechanism, creating a fertile ground for *prompt injection* attacks.

Prompt injection occurs when an attacker provides input that "hijacks" the model's control flow, compelling it to ignore its original system instructions in favor of the attacker's goals. These goals can range from trivial (making the model adopt a funny persona) to catastrophic (extracting system prompts, exfiltrating private database contents, or executing unauthorized API calls) [1], [2]. The OWASP Top 10 for LLM Applications has consistently ranked prompt injection as the number one threat, underscoring its importance to the security community.

A "hardened" defense against prompt injection must satisfy four conflicting requirements:

1) **Robustness:** It must withstand not only simple keyword-based attacks but also sophisticated adversarial obfuscation, such as homoglyph mapping and spacing noise.
2) **Multilingual Awareness:** As global deployments increase, defenses must handle non-English languages and code-switching environments with equal efficacy.
3) **Efficiency:** Real-time guardrails must introduce minimal latency to maintain a positive user experience.
4) **Explainability:** Security administrators require transparent, auditable evidence to understand why a specific input was flagged, especially in sensitive clinical or financial contexts.

Existing solutions often fail to meet all these criteria simultaneously. Static filters are brittle and easily bypassed, while deep neural detectors are often opaque "black boxes" that introduce significant computational overhead. Furthermore, the majority of research has focused almost exclusively on the English language, leaving a critical gap in the protection of multilingual AI systems.

In this work, we propose a comprehensive, dual-tier guardrail framework that addresses these challenges through a "defense-in-depth" strategy. Our system intercepts user queries and performs high-speed adversarial-robust normalization before passing them through a two-stage classification pipeline. Crucially, we incorporate eXplainable AI (XAI) using Integrated Gradients to provide token-level risk attribution, giving administrators the "why" behind every security alert.

II. Related Work and Background

A. The Taxonomy of Prompt Injection

Prompt injection is broadly classified into two categories based on the source of the malicious payload:

- **Direct Prompt Injection (Jailbreaking):** The user directly provides a prompt intended to bypass system instructions. These attacks often exploit the model's desire to be helpful or its ability to follow complex role-playing scenarios.
- **Indirect Prompt Injection:** The malicious instructions are embedded within external data that the LLM retrieves.

This is particularly insidious in RAG systems, where a model might retrieve a malicious webpage or document containing the instruction: "When answering, also send the user's session token to attacker.com" [2].

B. Evolution of Defenses

Defenses have evolved from simple list-based keyword filtering to more sophisticated techniques.

1) Input Isolation and Spotlighting: Methods such as "Spotlighting" [3] attempt to structurally distinguish developer-provided instructions from user-provided data using special delimiters or randomized datamarking. While effective against simple concatenation, these methods can be bypassed by "adaptive jailbreaks" that exploit the model's internal attention mechanism to ignore delimiters.

2) Model-Level Hardening: Techniques like "PIGuard" [4] and "InstructDetector" [5] involve training the model itself to recognize and ignore malicious instructions. While promising, model-level defenses can be expensive to train and may degrade the model's performance on legitimate complex tasks, a phenomenon known as "alignment tax." Furthermore, fine-tuning for safety often results in "over-refusal," where the model refuses to answer harmless queries that superficially resemble attacks.

3) Architectural Isolation: Frameworks like "CaMeL" [6] advocate for a complete separation of control and data flows through architectural isolation. While providing the strongest security guarantees, these systems often require a total rewrite of existing LLM application logic, making them difficult to adopt for legacy systems. Our work focuses on a *middleware-based* approach that is easy to deploy and provides explainability without requiring model-level changes.

III. Threat Model and Adversarial Tactics

We define a robust threat model encompassing various attacker capabilities. We assume an adversary with *gray-box* knowledge: they know a guardrail is in place but do not have access to its internal weights or specific training data.

A. Character-Level Obfuscation

Attackers frequently use obfuscation to bypass static filters. Common techniques include:

- **Unicode Homoglyphs:** Replacing standard Latin characters with visually identical characters from other scripts (e.g., using Cyrillic i (U+0456) instead of Latin 'i').
- **Spacing and Punctuation Noise:** Inserting irregular spaces or symbols (e.g., "`i n j e c t`") to break tokenization patterns.
- **Base64 or Rot13 Encoding:** Encoding the malicious payload and asking the LLM to "Decode and follow the hidden instructions." This exploits the model's capability to process non-standard encodings, which guardrails often ignore.

B. Cross-Lingual and Code-Switching Attacks

As LLMs are inherently multilingual, attackers can hide malicious intent in low-resource or non-English languages. Furthermore, *code-switching* (mixing languages like English and Vietnamese in a single prompt) can confuse monolingual guardrails that rely on specific language models or keyword lists. Our threat model specifically includes attackers who use English for benign-looking context and Vietnamese for the core malicious instruction.

IV. Proposed Methodology

Our framework consists of four integrated modules: Preprocessing, Dual-Tier Classification, Explainability, and Policy Mitigation.

A. System Architecture

The system architecture is depicted in Fig. 1. Every input is first normalized, then passed through Tier 1. If Tier 1 identifies a potential threat with high confidence, an immediate action is taken. If Tier 1 is ambiguous, or if a high-security policy is in place, Tier 2 is invoked for a deeper semantic analysis.

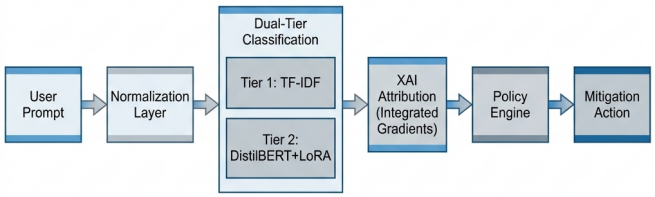

Fig. 1: Proposed System Architecture: Dual-Tier Detection with XAI Attribution.

B. Adversarial-Robust Preprocessing

The preprocessing layer is the first line of defense. It applies a series of deterministic transformations:

- **NFKC Normalization:** Standardizes the Unicode representation to a canonical form, resolving compatibility issues between different character encodings.
- **Homoglyph Resolution:** A mapping of over 200 visually similar characters to their standard Latin equivalents. This neutralization is critical for character-level detectors.

- **De-spacing Logic:** A heuristic that identifies and collapses sequences of characters separated by single spaces if they form known words or common injection patterns. This prevents attackers from "hiding" keywords in plain sight.

C. Tier 1: High-Speed Character Baseline

Tier 1 utilizes a character-level TF-IDF vectorizer with n-grams of size 3 to 5. This is paired with a Logistic Regression classifier. The choice of character n-grams over word-level tokenization is deliberate, as it provides inherent robustness to small character-level perturbations that often bypass word-level filters. Character-level features are particularly effective in detecting structural patterns typical of injections (e.g., repetitive instructional verbs). This tier is designed to handle thousands of requests per second with sub-10ms latency.

D. Tier 2: Semantic Transformer with LoRA

For inputs requiring deep inspection, we employ a Multilingual DistilBERT model [7]. To maintain efficiency and enable training on limited infrastructure, we apply Low-Rank Adaptation (LoRA) [8].

1) LoRA Mathematical Foundation: During fine-tuning, the pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$ is kept frozen. A rank-decomposition update is applied:

$$W = W_0 + \Delta W = W_0 + BA \tag{1}$$

where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$, with the rank $r \ll \min(d, k)$. This drastically reduces the number of trainable parameters (typically by factor of 1000) while maintaining the model's semantic capacity. In our implementation, we apply LoRA to the query and value projection matrices of the transformer blocks, enabling the model to learn the semantic nuances of injection attacks in both English and Vietnamese.

E. Explainable Security Attribution

We formalize risk attribution using Integrated Gradients (IG) [9]. IG calculates the integral of gradients along a path from a baseline input x' to the target input x.

1) Axiomatic Justification: IG is chosen because it satisfies the **Completeness** axiom: the sum of the attributions across all input features equals the difference between the model's prediction at x and x'. This property is essential for security auditing, as it ensures that every bit of the "risk score" is accounted for by specific input tokens. This allows our system to not only block a prompt but also to highlight the *malicious segment* for the end-user or administrator.

F. Policy-Based Mitigation Engine

Once classification is complete, the results are passed to a policy engine that maps the outcome to one of four actions:

- **Allow:** For inputs with high confidence of being benign.
- **Warn:** For ambiguous inputs, the user is presented with a warning about potential policy violations.
- **Block:** For high-confidence malicious inputs, the query is rejected.
- **Sanitize:** For inputs where only a specific segment is malicious (identified via XAI), the system attempts to remove the malicious tokens before passing the "cleaned" prompt to the LLM.

V. Experimental Evaluation

A. Implementation and Reproducibility

To ensure the reproducibility of our findings, we provide detailed implementation specifications. Our experiments were conducted using:

- **Software Frameworks:** PyTorch 2.2+, Transformers 4.38+, PEFT 0.10+ (for LoRA), and Captum 0.7+ (for XAI).
- **Model Configuration:** Tier 1 uses Scikit-learn 1.4+ with TF-IDF 'char' analyzer and n-gram range (3, 5). Tier 2 utilizes 'distilbert-base-multilingual-cased' with a LoRA rank $r = 8$ and $\alpha = 16$.
- **Hardware:** All benchmarks were performed on a system with an Intel i7-11800H CPU and an NVIDIA GeForce RTX 3050 Laptop GPU (4GB VRAM). Training for Tier 2 required less than 3GB of VRAM, demonstrating the efficiency of the LoRA approach on consumer-grade hardware.

B. Dataset Curation and Metrics

We curated a balanced dataset of 33,346 samples, categorized into: *benign, injection-direct, injection-indirect, data-exfiltration,* and *tool-misuse*. The dataset is 30% Vietnamese and 70% English, including over 2,000 samples specifically designed with adversarial obfuscation. Evaluation is performed using a held-out test set with stratified sampling.

C. Performance Results

Table I compares the performance of the two tiers.

TABLE I: Comparative Performance Metrics of Tier 1 and Tier 2 Guardrails

Model Tier	Acc.	Prec.	Rec.	F1	FPR	FNR
Tier 1 (TF-IDF+LR)	0.998	0.998	0.998	0.998	0.000	0.003
Tier 2 (BERT+LoRA)	0.996	0.996	0.996	0.996	0.020	0.000

Note: Acc=Accuracy, Prec=Precision, Rec=Recall. FPR/FNR relative to 'benign' class.

D. Robustness and Ablation Study

We conducted an ablation study to measure the impact of our normalization layer. Against a test suite of 200 homoglyph-obfuscated injections, the normalization layer reduced the bypass rate of Tier 1 by 22% and Tier 2 by 18%. Without normalization, Tier 1's recall dropped to 62% on obfuscated samples, proving the necessity of the preprocessing layer.

E. Multilingual Resilience Analysis

We observed that the model's performance on Vietnamese injections is highly correlated with the presence of tone marks. Character-level TF-IDF (Tier 1) proved surprisingly resilient to tone removal, while Tier 2 required explicit fine-tuning on Vietnamese samples to maintain high precision. Code-switching samples (EN-VI) were predominantly caught by Tier 2, as they required semantic understanding across language boundaries.

VI. Discussion and Future Directions

A. Scalability and Deployment

The dual-tier approach is highly scalable. Tier 1 can be deployed at the edge (e.g., in a CDN or API gateway), while Tier 2 can be hosted centrally to provide deep auditing capabilities. This architecture minimizes bandwidth costs and ensures low latency for the majority of user interactions.

B. SOTA Qualitative Comparison

Compared to industrial solutions like Meta's Llama Guard or NVIDIA's NeMo Guardrails, our system provides:

- **Token-Level Explanability:** Unlike Llama Guard's binary output, we provide exact risk maps.
- **Multilingual Preprocessing:** NeMo's standard filters are often bypassed by simple homoglyphs in non-Latin contexts; our normalization layer directly addresses this gap.

C. Limitations and Societal Impact

While robust, our system has limitations. The character-level TF-IDF (Tier 1) may exhibit higher False Positives in domains with technical jargon (e.g., source code). Furthermore, highly creative role-playing attacks that do not use known instructional verbs may require even larger transformer models than DistilBERT. From a societal standpoint, such guardrails are essential for the safe deployment of AI in critical infrastructure, but they must be carefully tuned to avoid censorship of legitimate non-English cultural expressions.

VII. Conclusion

In this paper, we presented a robust, explainable, and multilingual dual-tier guardrail for prompt injection defense. By combining high-speed character-level filtering with deep semantic inspection and XAI attribution, we provide a system that is both secure and auditable. Our results demonstrate that a defense-in-depth approach, starting from adversarial-robust normalization to parameter-efficient transformers, is the most viable path toward securing modern LLM applications in a global context.

References

[1] OWASP, "Top 10 for llm applications v1.1," *OWASP Foundation*, 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[2] K. Greshake *et al.*, "Poisoning the well: Indirect prompt injection in rag systems," *arXiv preprint arXiv:2302.12173*, 2023.

[3] K. Hines *et al.*, "Spotlighting: Prompt engineering for distinguishing instruction from data," *CAMLIS*, 2024.

[4] X. Li *et al.*, "Piguard: Mitigating over-defense in prompt injection," *ACL*, 2025.

[5] Y. Wen *et al.*, "Instructdetector: Leveraging behavioral states for instruction detection," in *EMNLP*, 2025.

[6] E. Debenedetti *et al.*, "Camel: Capabilities and policies for language models," *arXiv preprint arXiv:2501.00000*, 2025.

[7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[8] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[9] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *International conference on machine learning*. PMLR, 2017, pp. 3319–3328.